SQL Basics from Node

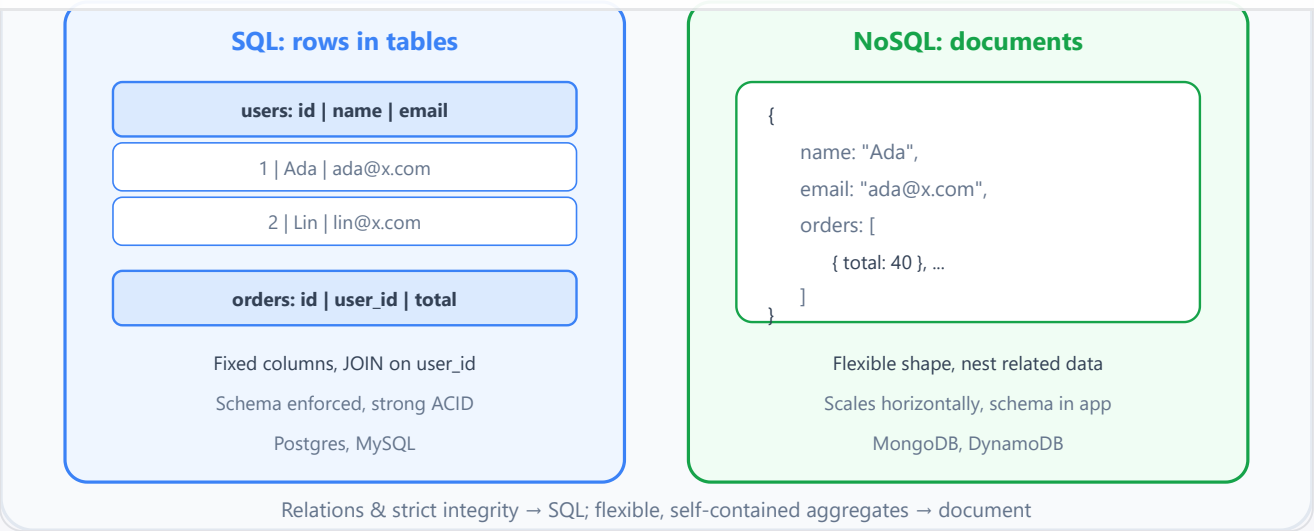
Why it matters

Not every backend should use MongoDB. A huge share of the industry — banking, e-commerce, anything with money, reporting, or strict relationships — runs on **relational (SQL) databases** like PostgreSQL and MySQL. As a Node developer you will absolutely be asked to talk to one, usually through the `pg` (node-postgres) or `mysql2` driver. And in interviews, "SQL vs NoSQL — which would you pick and why?" is one of the most common system-design openers there is. A weak answer ("Mongo is faster") tanks the conversation; a strong one shows you understand tradeoffs.

SQL also forces you to confront things a document store lets you gloss over: schemas as contracts, JOINs across tables, foreign keys, and real ACID transactions. Even if your day job is Mongo, understanding the relational model makes you a sharper data modeler everywhere. This topic gives you the working knowledge to query Postgres from Node safely and to reason about which database fits a problem.

The concept, simply

A relational database stores data in **tables** made of **rows** and fixed **columns**, each column with a declared type. Tables relate to each other through **keys**: a *primary key* uniquely identifies a row, and a *foreign key* in one table points at the primary key of another. You retrieve and combine data with **SQL** (Structured Query Language) — a declarative language where you describe *what* you want and the database's query planner figures out *how*.



SQL splits data into typed tables joined by keys; document stores nest related data into one flexible aggregate.

The essential contrast with NoSQL: SQL enforces a **fixed schema** and pushes relationships into **JOINs** resolved at query time, giving you strong consistency and integrity. Document stores keep related data together and stay flexible, trading some integrity for scale and developer speed.

How it works (deep)

SQL has a handful of core statement types. `SELECT` reads (with `WHERE` to filter, `ORDER BY` to sort, `LIMIT` / `OFFSET` to page, `GROUP BY` plus aggregates like `COUNT` / `SUM` to summarise). `INSERT`, `UPDATE`, and `DELETE` write. `CREATE TABLE` and `ALTER TABLE` define structure. The declarative nature matters: you never write loops; you state a set-based condition and the planner picks indexes and join strategies.

A **JOIN** combines rows from two tables on a matching condition. `INNER JOIN` keeps only rows with a match on both sides. `LEFT JOIN` keeps every row from the left table and fills `NULL` where the right side has no match (essential for "users and their orders, including users with zero orders"). `RIGHT` and `FULL OUTER` joins mirror that. Understanding INNER vs LEFT is a near-guaranteed interview question because the wrong one silently drops rows.

From Node, you connect with a driver. `pg` gives you a `Pool` (recommended) whose `pool.query()` borrows a connection, runs your SQL, and returns it automatically. The

array of values; the driver sends the query text and the data *separately*, so a value like `' OR 1=1 --` is treated as data, not code. This is how you prevent **SQL injection**, the most classic and dangerous web vulnerability.

When to use which? Reach for SQL when you have well-structured data with clear relationships, need strong consistency and multi-row ACID transactions (money, inventory, bookings), do a lot of ad-hoc reporting/aggregation, or must guarantee referential integrity. Reach for a document store when your data is self-contained aggregates, the schema evolves fast, you need to scale writes horizontally across shards, or the shape is naturally hierarchical (a product with wildly varying attributes). Modern reality: many systems use **both** (polyglot persistence), and Postgres even has a strong `jsonb` type that blurs the line.

Code examples

Setting up a pool and running a safe, parameterised query with node-postgres:

JAVASCRIPT Copy

```
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.PGHOST,
  database: 'shop',
  max: 10,                  // pool size
  idleTimeoutMillis: 30000
});

// PARAMETERISED - safe from SQL injection.
async function findUser(email) {
  const { rows } = await pool.query(
    'SELECT id, name, email FROM users WHERE email = $1',
    [email]                 // values passed separately from SQL text
  );
  return rows[0] ?? null;  // rows is an array
}
```

The SQL itself — schema, insert, and a JOIN across two tables:

Node.js Backend Kit

SQL

Copy

```sql
CREATE TABLE users (
  id    SERIAL PRIMARY KEY,
  name  TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL
);

CREATE TABLE orders (
  id      SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),  -- foreign key
  total   NUMERIC(10,2) NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Every user with their order count (LEFT JOIN keeps users with 0 orders)
SELECT u.id, u.name, COUNT(o.id) AS order_count, COALESCE(SUM(o.total), 0)
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name
ORDER BY spent DESC
LIMIT 20;
```

INSERT returning the new row, plus UPDATE and DELETE from Node:

JAVASCRIPT

Copy

```javascript
// INSERT ... RETURNING gives you the generated id back in one round trip
const { rows } = await pool.query(
  'INSERT INTO users (name, email) VALUES ($1, $2) RETURNING id, created_a
  ['Ada', 'ada@x.com']
);
const newId = rows[0].id;

await pool.query(
  'UPDATE orders SET total = total * 1.1 WHERE user_id = $1',
  [newId]
);
```

The wrong way — string concatenation — and why it is dangerous:

JAVASCRIPT Copy

```
// NEVER DO THIS - open to SQL injection:
const email = req.query.email; // e.g. "x' OR '1'='1"
const bad = await pool.query(
  `SELECT * FROM users WHERE email = '${email}'` // injectable!
);
// The attacker's input becomes part of the SQL and can dump the table.
// Always use $1 placeholders instead (see findUser above).
```

Common mistakes & gotchas

- **String-concatenating user input into SQL.** This is the #1 web vulnerability (SQL injection). Always use `$1` / `?` parameterised placeholders.
- **Using `INNER JOIN` when you meant `LEFT JOIN`.** Inner join silently drops parent rows that have no children (users with zero orders vanish from the report).
- **Forgetting `WHERE` on `UPDATE` / `DELETE`.** `DELETE FROM users;` wipes the whole table. There is no undo without a backup.
- **Assuming `rows` is a single object.** `pg` returns `{ rows: [...] }` — an array. Read `rows[0]` for a single record.
- **Mishandling `NULL`.** `column = NULL` is never true; use `IS NULL` / `IS NOT NULL`. Aggregates like `SUM` ignore NULLs, so wrap with `COALESCE`.
- **Selecting `*` everywhere.** It fetches columns you do not need, breaks when the schema changes, and prevents covering indexes. Name your columns.
- **N+1 queries.** Looping over users and querying orders per user is slow; do one JOIN or an `IN (...)` query.

Best practices

- Use a `Pool`, not one-off `Client` connections per request, and let it manage reuse (covered in the next topic).
- Prefer explicit column lists over `SELECT *`; it is faster, clearer, and index-friendly.
- Add indexes on foreign keys and columns you filter/sort on; check plans with `EXPLAIN ANALYZE`.
- Use `INSERT ... RETURNING` to get generated ids in one round trip instead of a follow-up SELECT.
- Push filtering, sorting, and aggregation into SQL rather than pulling rows into Node and processing them — the database is far better at it.
- Pick the database from the problem: relational integrity and transactions → SQL; flexible self-contained aggregates and horizontal scale → document store; often both.

⭐ Interview lens

Q: SQL vs NoSQL — when would you choose each?

SQL for structured, related data needing strong consistency, multi-row ACID transactions, and rich ad-hoc queries/joins (finance, inventory, bookings). Document/NoSQL for self-contained aggregates, rapidly evolving schemas, hierarchical data, and horizontal write scale. The mature answer notes it is not either/or — polyglot persistence is common, and Postgres `jsonb` gives document-style flexibility inside a relational engine.

Q: What is SQL injection and how do you prevent it?

An attack where user input is concatenated into SQL and executed as code, letting an attacker read or destroy data. Prevention is parameterised queries: send the SQL text and the values separately (`$1` in pg, `?` in mysql2) so values are always treated as data. ORMs/query builders do this for you; never build SQL with template strings around user input.

INNER JOIN returns only rows with a match in both tables. LEFT JOIN returns every row from the left table and NULLs where the right has no match. The classic trap: "list all users and their order totals" — INNER JOIN silently omits users with zero orders, so you need LEFT JOIN (plus `COALESCE` to turn NULL sums into 0).

Q: What is a foreign key and what does it give you?

A column that references another table's primary key, declaring a relationship the database enforces — you cannot insert an order for a non-existent user, and (with cascade rules) deletes can propagate. It provides *referential integrity*, a guarantee document stores generally leave to application code.

Q: Why prefer a Pool over creating a Client per request?

Opening a TCP connection and authenticating to Postgres is expensive (tens of milliseconds). A pool keeps a set of connections open and hands them out, so requests reuse warm connections instead of paying setup cost each time, and it caps total connections so you do not overwhelm the database. Per-request clients exhaust connection limits under load.

Q: How does NULL behave in SQL comparisons and aggregates?

NULL means "unknown", so any comparison with it (`= NULL`, `<> NULL`) yields NULL (not true) — you must use `IS NULL` / `IS NOT NULL`. Aggregates like `COUNT(col)`, `SUM`, and `AVG` skip NULLs, which is why a sum over no rows returns NULL and you wrap it in `COALESCE( ... , 0)`.

Cheat-card

- **Model:** tables → rows → typed columns; primary key + foreign key relate tables.
- **Drivers:** `pg` (Postgres, `$1` params), `mysql2` (`?` params).
- **Safety:** parameterised queries always — never concatenate user input.

- **NULL:** use `IS NULL`; wrap aggregates in `COALESCE`.
- **Choose SQL** for integrity + transactions; **document** for flexible aggregates + scale.

QUICK CHECK — no marks, just for you.

1. What is the correct defense against SQL injection from Node?

- ◯ Escaping quotes in user input by hand before concatenating it
- ◯ Running every query inside a transaction
- ◯ Parameterized queries that send the SQL text and the values separately (e.g. $1 in pg, ? in mysql2)
- ◯ Using SELECT * so the query shape never changes

Check answer

2. For "list all users and their order totals, including users with zero orders," which join do you need?

- ◯ INNER JOIN, because it is faster and returns every user
- ◯ RIGHT JOIN from users to orders
- ◯ No join; a single SELECT on users is enough
- ◯ LEFT JOIN, because INNER JOIN silently drops users who have no matching orders

Check answer

3. How does NULL behave in a SQL comparison like column = NULL?

It is never true, so you must use IS NULL / IS NOT NULL instead

- It evaluates to true whenever the column is empty
- It raises a syntax error and aborts the query
- It matches every row because NULL equals everything

Check answer

Previous
Schema Design & Data Modeling

Next
Connection Pooling

LEARN MORE

ARTICLES & DOCS

- node-postgres — PostgreSQL client for Node.js
  node-postgres.com
- The SQL Language (PostgreSQL Tutorial, Chapter 2)
  www.postgresql.org
- MySQL 8.0 Reference Manual: Tutorial
  dev.mysql.com

VIDEOS



SQL Tutorial - Full Database Course for Beginners
freeCodeCamp.org

Hand-picked references for this lesson.